Abdellah BENSALEM et Mohamed TROMBATI

C++ pour les Mathématiques Appliquées

Compte Rendu de Travaux Pratiques

Simulation de Particules en C++

TP2 • TP3 • TP4

Module :	C++ pour les Mathématiques Appliquées
Établissement :	Grenoble INP – MMIS
Année :	2024–2025

Table des matières

Introduction **2**

1 TP2 : Structures de donnees et simulation de particules **3**
1.1 Objectifs . . . 3
1.2 Classe Particule . . . 3
1.3 Algorithme de Störmer-Verlet . . . 3
1.4 Simulation du système solaire . . . 4
1.5 Comparaison `vector` vs `list` . . . 4

2 TP3 : Operateurs et structures de donnees avancées **5**
2.1 Objectifs . . . 5
2.2 Classe Vecteur . . . 5
2.2.1 Attributs et encapsulation . . . 5
2.2.2 Operateurs implémentés . . . 5
2.3 Classe Particule . . . 5
2.4 Classe Univers . . . 5
2.4.1 Description . . . 5
2.4.2 Calcul des forces gravitationnelles . . . 6
2.5 Tests unitaires . . . 6
2.6 Benchmarks de performance . . . 6

3 TP4 : Decoupage spatial et potentiel de Lennard-Jones **7**
3.1 Objectifs . . . 7
3.2 Potentiel de Lennard-Jones . . . 7
3.2.1 Formule . . . 7
3.2.2 Force dérivée . . . 7
3.2.3 Rayon de coupure . . . 7
3.3 Grille de cellules . . . 7
3.3.1 Principe . . . 7
3.3.2 Classe Cellule . . . 8
3.3.3 Mise a jour des cellules . . . 8
3.4 Optimisations implémentées . . . 8
3.5 Tests unitaires . . . 8
3.5.1 Infrastructure de test . . . 8
3.5.2 Test de la valeur analytique des forces . . . 9
3.5.3 Test de conservation de la quantite de mouvement . . . 9
3.6 Export VTK pour ParaView . . . 9
3.6.1 Format . . . 9
3.7 Simulation de collision . . . 9
3.7.1 Parametres . . . 9
3.7.2 Description . . . 10

Conclusion **11**

Introduction

Ce compte rendu présente les travaux pratiques réalisés dans le cadre du cours *C++ pour les mathématiques appliquées*. L'objectif global est de développer, par étapes successives, une simulation de particules capable de modéliser des interactions physiques.

Chaque TP enrichit le projet d'une couche supplementaire :

— **TP2** introduit la classe `Particule` et l'algorithme de Störmer-Verlet pour la simulation du système solaire ;

— **TP3** structure le code en classes indépendantes (`Vecteur`, `Particule`, `Univers`) avec surcharge d'opérateurs ;

— **TP4** introduit un découpage spatial par grille de cellules, le potentiel de Lennard-Jones, et l'export VTK pour ParaView.

1 TP2 : Structures de donnees et simulation de particules

1.1 Objectifs

Le TP2 a pour objectif de créer une première simulation de système de particules en C++, d'explorer les conteneurs standards de la STL (`std::vector` et `std::list`), et d'implémenter l'algorithme de Störmer-Verlet pour l'intégration numérique des équations du mouvement.

1.2 Classe Particule

La classe `Particule` encapsule toutes les propriétés d'une particule physique. Les attributs sont privés et accessibles via des getters et setters :

- **Position** `(x, y, z)` : coordonnees spatiales en double precision
- **Vitesse** `(vitessex, vitessey, vitessez)` : composantes du vecteur vitesse
- **Force** `(forcex, forcey, forcez)` : force resultante appliquee à la particule
- **Masse** : scalaire `double`, avec validation (exception si negative où nulle)
- **Identifiant** et **categorie** : metadata de la particule

Un soin particulier est apporte à la validation : le setter `setMasse()` leve une exception `std::invalid_argument` si la masse est inférieure où égale a zero.

```
void setMasse(double m) {
    if (m <= 0)
        throw std::invalid_argument("La masse doit etre
           positive.");
    masse = m;
}
```

Listing 1 – Validation de la masse dans setMasse()

1.3 Algorithme de Störmer-Verlet

L'algorithme de Störmer-Verlet (vitesse-Verlet) est une méthode d'intégration numérique d'ordre 2. Il se déroule en trois étapes par pas de temps Δt :

$$\mathbf{x}(t+\Delta t) = \mathbf{x}(t) + \mathbf{v}(t)\,\Delta t + \frac{1}{2}\mathbf{a}(t)\,\Delta t^2 \tag{1}$$

$$\mathbf{F}(t+\Delta t) = \text{calculerForces}\Big(\mathbf{x}(t+\Delta t)\Big) \tag{2}$$

$$\mathbf{v}(t+\Delta t) = \mathbf{v}(t) + \frac{1}{2}\Big(\mathbf{a}(t) + \mathbf{a}(t+\Delta t)\Big)\,\Delta t \tag{3}$$

Cet algorithme conservé mieux l'énergie totale du système sur de longues simulations par rapport à l'intégrateur d'Euler simple, ce qui est crucial pour les orbites planetaires.

1.4 Simulation du système solaire

La simulation comporte quatre corps :

Corps	Masse	Vitesse initiale
Soleil	1.0	$(0, 0, 0)$
Terre	3×10^{-6}	$(-1, 0, 0)$
Jupiter	9.55×10^{-4}	$(-0.425, 0, 0)$
Halley	10^{-14}	$(0, 0.0296, 0)$

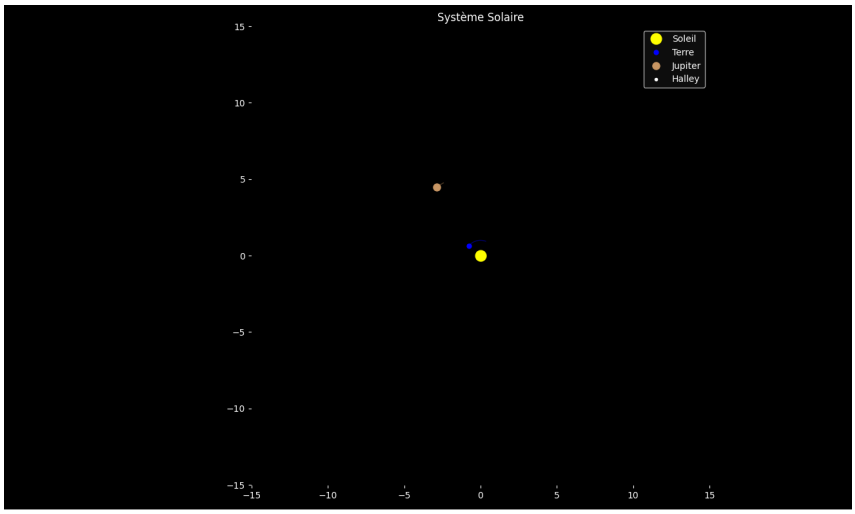

FIGURE 1 – Simulation de système solaire

Les parametres sont $\Delta t = 0.015$ et $t_{\text{end}} = 468.5$. Les positions sont sauvegardees dans `positions.txt`. La visualisation a été faite par python dans notre répertoire de TP, on avait les données en utilisant pandas.

1.5 Comparaison `vector` vs `list`

Une comparaison de performances est realisee pour l'insertion de N particules aleatoires :

Conteneur	Avantages	Inconvenients	Cas d'usage
`std::vector`	Acces aleatoire O(1), cache-friendly	Insertion en milieu O(N)	Simulation (accès par indice)
`std::list`	Insertion O(1) partout, pas de reallocation	Pas d'accès aleatoire, mauvaise localite memoire	Suppressions frequentes en milieu

Le `std::vector` s'avere systematiquement plus rapide pour notre usage car les particules sont accedees par indice a chaque pas de temps.

2 TP3 : Operateurs et structures de donnees avancées

2.1 Objectifs

Le TP3 restructure et enrichit le code du TP2 en separant le projet en plusieurs classes indépendantes avec leurs propres fichiers. Il introduit la classe `Vecteur` avec surcharge d'opérateurs et la classe `Univers` pour gerer un ensemble de particules.

2.2 Classe Vecteur

2.2.1 Attributs et encapsulation

Les attributs `x`, `y`, `z` sont privés. L'accès est gere par des getters `const` pour la lecture et des setters pour la modification.

2.2.2 Operateurs implémentés

Operateur	Description	Exemple
`operator+`	Addition vectorielle	`v1 + v2`
`operator-`	Soustraction vectorielle	`v1 - v2`
`operator*`	Multiplication par un scalaire	`v * 2.0`
`operator/`	Division par un scalaire	`v / m`
`operator+=`	Addition en place	`v += acc * dt`
`operator-=`	Soustraction en place	`force -= F`
`norm()`	Norme euclidienne	`rij.norm()`

2.3 Classe Particule

La classe `Particule` utilisé desormais trois instances de `Vecteur` (`position`, `vitesse`, `force`) au lieu de neuf scalaires séparés. Deux versions de getters sont fournies pour chaque vecteur :

```
// Getter const pour la lecture
const Vecteur& getPosition() const { return position; }

// Getter non-const : retourne une reference pour += et -=
Vecteur& getPosition() { return position; }
```

Listing 2 – Getters const et non-const de Particule

2.4 Classe Univers

2.4.1 Description

La classe `Univers` contient une collection de particules et gere la simulation gravitationnelle. Elle intègre un vecteur de particules, la dimension de l'espace (1D, 2D ou 3D), le temps courant t, et les méthodes `calcul_forces()`, `avancer()` et `afficher()`.

2.4.2 Calcul des forces gravitationnelles

L'interaction gravitationnelle entre chaque paire (i, j) est calculée avec un parametre de softening $\varepsilon = 0.05$ pour éviter les singularites a courte distance :

$$F_{ij} = \frac{G\, m_i\, m_j}{r_{ij}^3 + \varepsilon^2} \cdot \mathbf{r}_{ij}$$

La troisieme loi de Newton est exploitée en ne calculant chaque paire (i, j) qu'une seule fois (boucle avec $j > i$), divisant le nombre d'opérations par 2.

2.5 Tests unitaires

Des tests google sont écrits pour chaque méthode de la classe `Vecteur` (constructeurs, norme, les quatre opérations arithmetiques, les opérateurs cumulés), ainsi que pour la conservation de l'impulsion dans l'`Univers`, on peut lacer la commande ./tests pour les afficher tout, ou on peut les filtrer par gtest filter,par exemple pour lancer que les testes de particule seulement

2.6 Benchmarks de performance

La complexité du calcul des forces est $O(N^2)$, ce qui fait que la complexité se multiplie à chaque k par un facteur de 64. Les benchmarks montrent la croissance quadratique attendue :

k	$N = (2^k)^3$	Paires évaluées	Temps approximatif
3	512	130 816	∼12 ms
4	4 096	8 386 560	∼700 ms
5	32 768	536 854 528	∼47 s
6	262 144	>34 milliards	Prohibitif

Le facteur d'accélération apporte par Newton 3 est d'environ 2× par rapport à la version naïve. Pour aller plus loin, une structure spatiale hiérarchique (grille de cellules) devient indispensable, ce qui est l'objet du TP4.

3 TP4 : Decoupage spatial et potentiel de Lennard-Jones

3.1 Objectifs

Le TP4 introduit deux avancées majeures : le potentiel de Lennard-Jones comme modèle d'interaction, et un découpage de l'espace en grille de cellules pour réduire la complexité de $O(N^2)$ a quasi-linéaire grâce à un rayon de coupure.

3.2 Potentiel de Lennard-Jones

3.2.1 Formule

Le potentiel de Lennard-Jones decrit l'interaction entre deux particules en fonction de leur distance r_{ij} :

$$U(r_{ij}) = 4\varepsilon \left[\left(\frac{\sigma}{r_{ij}} \right)^{12} - \left(\frac{\sigma}{r_{ij}} \right)^{6} \right]$$

Le terme en r^{-12} modélisé la repulsion a courte distance et le terme en r^{-6} modélisé l'attraction a longue distance. Les parametres utilisés sont $\varepsilon = 5.0$ et $\sigma = 1.0$.

3.2.2 Force dérivée

La force sur la particule i est dérivée du potentiel global :

$$\mathbf{F}_{ij} = \frac{24\varepsilon}{r_{ij}^2} \left[\left(\frac{\sigma}{r_{ij}} \right)^{6} - 2 \left(\frac{\sigma}{r_{ij}} \right)^{12} \right] \mathbf{r}_{ij}$$

La distance d'equilibre est $r^* = 2^{1/6}\,\sigma \approx 1.122\,\sigma$, point où la force est nulle. En dessous de r^*, la force est repulsive ; au-dessus, elle est attractive.

3.2.3 Rayon de coupure

Au-dela de $r_{\text{cut}} = 2.5\,\sigma$, le potentiel est neglige :

$$U(r_{ij}) = \begin{cases} 4\varepsilon \left[\left(\frac{\sigma}{r_{ij}} \right)^{12} - \left(\frac{\sigma}{r_{ij}} \right)^{6} \right] & r_{ij} \leq r_{\text{cut}} \\ 0 & r_{ij} > r_{\text{cut}} \end{cases}$$

3.3 Grille de cellules

3.3.1 Principe

L'espace de simulation est découpé en une grille tensorielle de cubes. Le nombre de cellules dans chaque direction est :

$$n_{cd} = \left\lfloor \frac{L_d}{r_{\text{cut}}} \right\rfloor$$

Chaque cellule connait ses voisines (jusqu'à 27 en 3D). Pour calculer les forces d'une particule, on n'examine que les particules dans les cellules voisines, garantissant que toutes les particules a distance $\leq r_{\text{cut}}$ sont considerees.

3.3.2 Classe Cellule

La classe `Cellule` encapsule :

— La liste des indices des particules (`std::vector<int>`)

— Son centre geometrique (`Vecteur`)

— Un tableau précalculé des indices de ses cellules voisines (`std::array<int,27>`)

— Le nombre de voisins valides (`nb_voisins`)

Les attributs sont privés avec getters et setters. Les voisins sont précalculés à l'initialisation.

3.3.3 Mise a jour des cellules

A la fin de chaque pas de temps, `mettre_a_jour_cellules()` vide toutes les cellules puis réaffecte chaque particule a sa cellule courante selon sa position. Cette operation est $O(N)$ et est appelée deux fois par pas de temps (une avant chaque calcul de forces dans le schema Verlet).

3.4 Optimisations implémentées

Optimisation	Description	Gain estime
Newton 3	Chaque paire (i, j) calculée une seule fois	$2\times$
Grille + r_{cut}	Seules les cellules voisines sont examinees	$O(N^2) \to O(N)$
Calcul sur r^2	Evite le `sqrt()` pour la comparaison avec r_{cut}^2	$\sim 1.5\times$
Sans `pow()`	`sr2*sr2*sr2` au lieu de `pow(sr2,3)`	$\sim 2\times$
Compilation `-O2/-O3`	Optimisations du compilateur	5 a $10\times$

3.5 Tests unitaires

3.5.1 Infrastructure de test

Les tests sont organises en quatre fichiers indépendants :

— `test_vecteur.cxx` : constructeurs, norme, opérations arithmetiques

— `test_particule.cxx` : constructeur par defaut, setters/getters, modification par référence

— `test_univers.cxx` : dimensions de grille, affectation des cellules, voisins, conservation de la quantite de mouvement

— `test_force.cxx` : distance d'equilibre LJ, signe de la force, Newton 3, rayon de coupure, valeur analytique

3.5.2 Test de la valeur analytique des forces

Un test particulierement rigoureux compare la force calculée par simulation avec la valeur analytique exacte pour deux particules alignees sur l'axe X a distance $r = 1.1\,\sigma$:

```
double Fx_sim  = ps[0].getForce().getX();
double Fx_anal = force_lj_analytique(eps, sig, r);
check(approx(Fx_sim, Fx_anal, 1e-8),
      "Fx simulee == Fx analytique pour r=1.1*sigma");
```

Listing 3 – Verification de la valeur analytique de la force LJ

3.5.3 Test de conservation de la quantite de mouvement

Le test verifie que la somme des quantites de mouvement reste constante apres 10 pas de temps, confirmant que Newton 3 est bien applique :

$$\left| \sum_i m_i \mathbf{v}_i(t + 10\,\Delta t) - \sum_i m_i \mathbf{v}_i(t) \right| < 10^{-8}$$

3.6 Export VTK pour ParaView

3.6.1 Format

Chaque frame est sauvegardee dans un fichier `.vtp` (VTK PolyData XML) contenant :

- Les positions 3D de toutes les particules
- La norme de la vitesse (utilisée pour la colorisation dans ParaView)
- Le vecteur vitesse complet (3 composantes)
- L'identifiant de chaque particule

Les fichier sont stockés dans le répertpoire$\texttt{vtk}_{o}utputtréférencetoutesleslesframesavecleurinstanttempo$

3.7 Simulation de collision

3.7.1 Parametres

Parametre	Valeur	Description
N_{total}	8 000	1 600 (rouge) + 6 400 (bleu)
ε	5.0	Profondeur du potentiel LJ
σ	1.0	Distance caractéristique LJ
r_{cut}	$2.5\,\sigma$	Rayon de coupure
Δt	5×10^{-5}	Pas de temps
t_{end}	19.5	Duree totale
$\mathbf{v}_0$ (rouge)	$(0, -10, 0)$	Vitesse initiale du bloc rouge
d_{inter}	$2^{1/6}\,\sigma$	Distance d'equilibre LJ

3.7.2 Description

Un bloc rouge de 40×40 particules tombe avec une vitesse initiale de $v = (0, -10, 0)$ sur un bloc bleu stationnaire de 160×40 particules. La collision généré des vitesses elevees dans la zone d'impact, visualisees par une colorisation rouge dans ParaView.

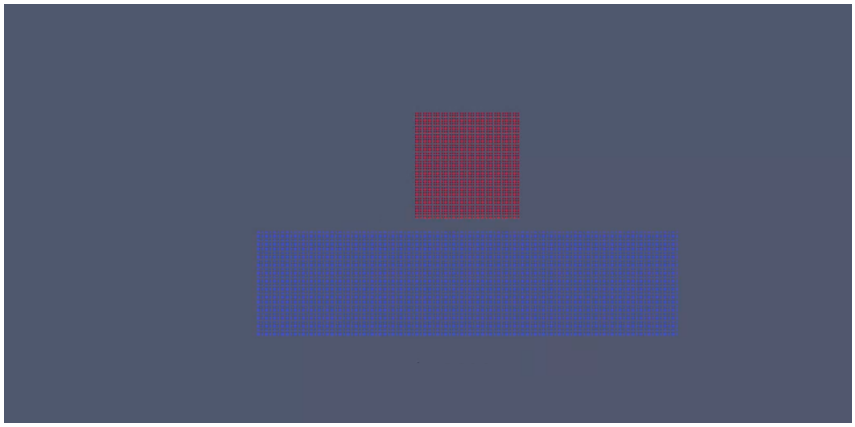

FIGURE 2 – Début de simulation

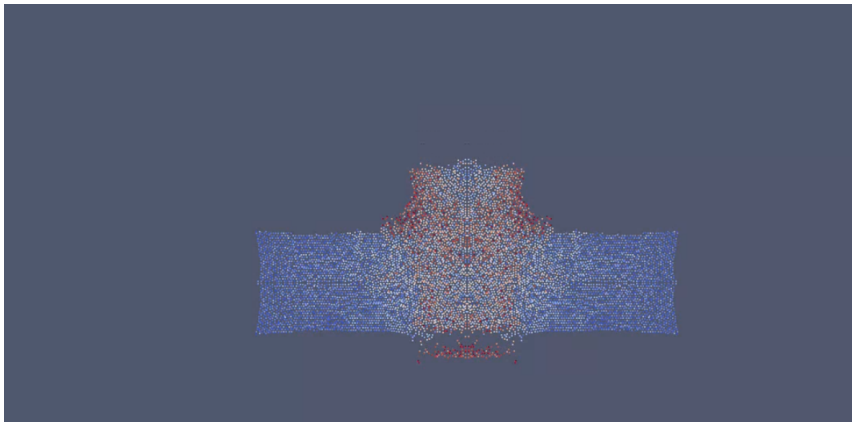

FIGURE 3 – Après quelques instansts

Conclusion

Ces trois travaux pratiques ont permis de construire progressivement une simulation de particules complète en C++ moderne.

Sur le plan **algorithmique**, l'évolution est significative :

- Le passage de l'intégrateur d'Euler a Störmer-Verlet garantit une conservation de l'énergie d'ordre 2.
- L'introduction de la grille de cellules réduit la complexité du calcul des forces de $O(N^2)$ à un comportement quasi-linéaire grâce au rayon de coupure r_{cut}.

Sur le plan **génie logiciel**, le code illustre les bonnes pratiques C++ :

- Encapsulation avec attributs privés et getters/setters
- Surcharge d'opérateurs pour la lisibilité des calculs physiques
- Separation des responsabilites entre classes
- Tests unitaires indépendants et répétables

La visualisation via ParaView avec le format VTK standard constitue un apport important : elle permet de valider visuellement les résultats et offre des outils de post-traitement scientifique (colorisation par vitesse, suivi temporel).